

Sistema de análisis digital de oscilaciones gestuales en pacientes con párkinson mediante inteligencia artificial

Esp. Ing. Alex Damián Barria Cárdenas

Maestría en Inteligencia Artificial

Director: PhD. Ing. Fernando Bernuy (Universidad Nacional de Chile)

Jurados:

Jurado 1 (pertenencia)

Jurado 2 (pertenencia)

Jurado 3 (pertenencia)

Ciudad de Buenos Aires, mayo de 2026

Resumen

En este trabajo se desarrolló un sistema de software que analiza grabaciones de video de personas que realizan ejercicios estandarizados de evaluación motora en pacientes con párkinson. La solución produce medidas cuantitativas del movimiento para complementar la valoración clínica con información objetiva y trazable. La enfermedad de Parkinson altera el control motor y suele evaluarse en consulta mediante la observación de esos ejercicios, lo que puede introducir subjetividad y dificultar el seguimiento fino de la evolución.

Para su implementación, se aplicaron contenidos de la maestría en inteligencia artificial, en particular visión por computadora, aprendizaje automático con modelos preentrenados, ingeniería de software y operaciones de aprendizaje automático.

Agradecimientos

Esta sección es para agradecimientos personales y es totalmente **OPCIONAL**.

Índice general

Resumen	I
1. Introducción general	1
1.1. Introducción general al tema	1
1.2. Contexto y motivación	2
1.3. Estado del arte	3
1.4. Objetivos del trabajo	4
1.4.1. Objetivo general	4
1.4.2. Objetivos específicos	4
2. Introducción específica	7
2.1. Frameworks y bibliotecas	7
2.2. Datasets y datos	8
2.3. Modelos preentrenados	10
2.4. Infraestructura	12
3. Diseño e implementación	15
3.1. Pipeline de datos	15
3.1.1. Ingestión de video y audio	15
3.1.2. Preprocesamiento de imagen	15
3.1.3. Calibración espacial	16
3.1.4. Fase estática y segmentación del ejercicio	17
3.2. Arquitectura de la solución analítica	18
3.2.1. Alcance del aprendizaje automático	19
3.2.2. Criterios de diseño y configuración	19
3.2.3. Contrato de datos entre etapas	20
3.2.4. Orquestación de la inferencia	21
3.2.5. Seguimiento temporal y señales derivadas	22
3.2.6. Métricas de movimiento implementadas	22
3.2.7. Agregación, exportación y trazabilidad	25
3.3. Integración e implementación del sistema	25
3.3.1. Diferencias entre ejecución por CLI y por API	25
3.3.2. Servicio HTTP y procesamiento en segundo plano	26
3.3.3. Manejo de errores y recuperación de sesión	26
3.3.4. Artefactos y consulta de resultados	27
3.3.5. Interfaz web	27
3.3.6. Despliegue local y en la nube	28
4. Ensayos y resultados	31
4.1. Pruebas automatizadas del pipeline	31
5. Conclusiones	33
5.1. Conclusiones generales	33

5.2. Próximos pasos	33
Bibliografía	35

Índice de figuras

1.1. Ejercicio de finger tapping utilizado en la evaluación clínica de movimientos finos de la mano.	2
2.1. Ejemplo ilustrativo de los puntos de referencia entregados por el modelo <code>HandLandmarker</code> de <code>MediaPipe</code>	11
3.1. Detección del tablero <code>ChArUco</code> para estimación de relación milímetros por píxel.	17
3.2. Serie temporal de la distancia pulgar-índice en una sesión de finger tapping.	18
3.3. Flujo funcional del sistema desde la entrada de video hasta reportes y visualizaciones.	21
3.4. Serie temporal de la distancia pulgar-índice durante una sesión completa.	24
3.5. Interfaz de carga de sesión y de visualización de resultados (imagen pendiente de incorporar).	28
3.6. Arquitectura de despliegue del prototipo en Google Cloud Platform.	29

Índice de tablas

1.1. Enfoques frecuentes para cuantificar el movimiento en Parkinson y contextos afines.	4
2.1. Bibliotecas y herramientas de terceros utilizadas en el desarrollo del sistema, agrupadas por capa funcional.	8
2.2. Caracterización resumida del conjunto de grabaciones de video utilizado en el desarrollo y la verificación del sistema.	10
2.3. Modelos preentrenados de terceros incorporados al sistema.	12
2.4. Servicios gestionados de GCP seleccionados para la prueba de concepto.	13
3.1. Etapas del pipeline de procesamiento de video y componentes asociados.	18
3.2. Artefactos generados por sesión y su consumo en la aplicación.	27

Dedicado a... [OPCIONAL]

Capítulo 1

Introducción general

En este capítulo se contextualiza la enfermedad de Parkinson, la manera habitual de evaluar sus signos motores y se expone la motivación para incorporar análisis automático de video. También se sintetiza el estado del arte en visión por computadora aplicada a trastornos del movimiento y se enuncian los objetivos del trabajo.

1.1. Introducción general al tema

La enfermedad de Parkinson es un trastorno neurodegenerativo frecuente que compromete principalmente el sistema motor. Entre sus manifestaciones se cuentan la bradicinesia, la rigidez, el temblor en reposo y alteraciones de la marcha y el equilibrio. La gravedad y la evolución de estos signos condicionan la calidad de vida del paciente y las decisiones terapéuticas del profesional de la salud.

En la práctica neurológica se emplean escalas validadas que formalizan la exploración motora. Una referencia ampliamente adoptada es la parte motora de la escala unificada de la Sociedad Internacional de Parkinson y trastornos del movimiento (MDS por sus siglas en inglés), conocida en la literatura como MDS-UPDRS parte III, que guía la observación de tareas como movimientos de dedos, postura y marcha [1]. Esas puntuaciones son útiles para comparar pacientes y momentos en el tiempo, pero dependen de la pericia del examinador y del contexto de la consulta, factores que introducen variabilidad entre observadores y entre sesiones.

En los últimos años creció el interés por registrar la exploración en video, ya sea en forma presencial o remota, y por apoyar la lectura clínica con algoritmos que midan trayectorias, ritmo y amplitud a partir de las imágenes [1, 2]. Esa línea de trabajo apunta a obtener descriptores cuantitativos reproducibles a partir de grabaciones convencionales, sin reemplazar al médico pero ofreciéndole un respaldo objetivo para el seguimiento.

Para situar visualmente el tipo de tareas que los profesionales solicitan durante la exploración motora, la figura 1.1 muestra un ejemplo de finger tapping: toques rítmicos entre el pulgar y el índice, frecuentemente incluidos en protocolos como la parte motora de la escala MDS-UPDRS. En la imagen se observa la mano en la postura típica de la prueba, que permite al examinador valorar amplitud, regularidad y fatiga del movimiento. Esa misma secuencia puede registrarse en video para analizarla de forma cuantitativa con herramientas de visión por computadora, como se desarrolla en los capítulos posteriores.

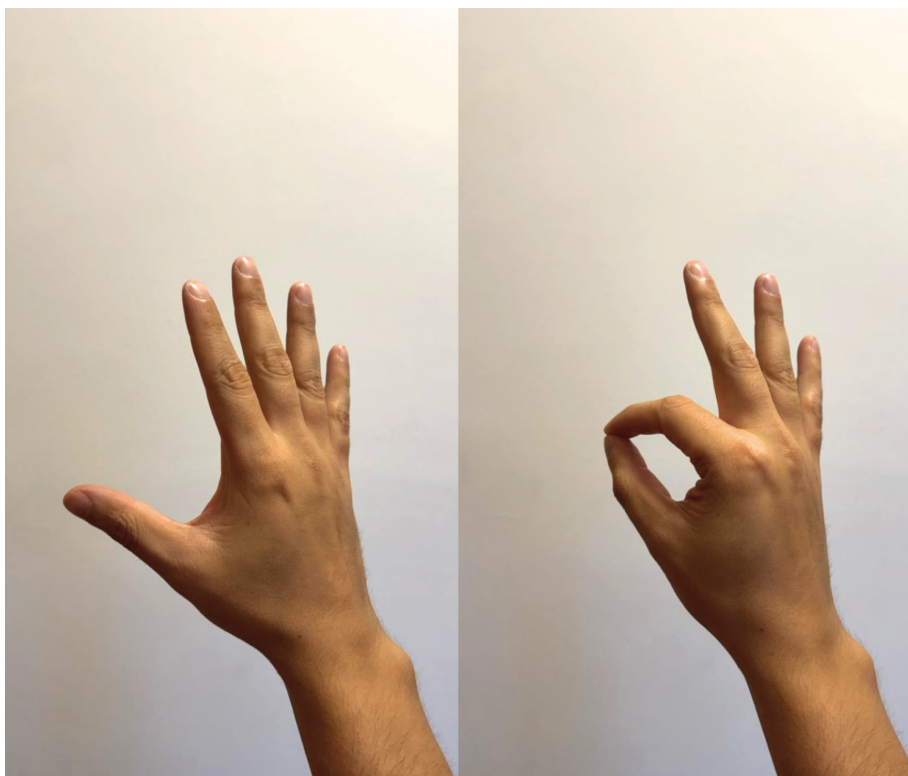


FIGURA 1.1. Ejercicio de finger tapping utilizado en la evaluación clínica de movimientos finos de la mano.

1.2. Contexto y motivación

La motivación del trabajo fue construir una herramienta que, a partir de videos de ejercicios motores, estimara la pose corporal del paciente y derivara métricas asociadas a oscilaciones y regularidad del movimiento, de modo de reducir la dependencia exclusiva de la impresión visual humana para apreciar cambios leves o graduales.

La inteligencia artificial y la visión por computadora resultaron adecuadas porque permiten procesar cada fotograma de manera uniforme, conservar el detalle temporal entre cuadros consecutivos y transformar coordenadas articulares en magnitudes interpretables por el personal de salud. Se priorizó un enfoque basado en modelos preentrenados de estimación de pose, alineado con el alcance acordado para el proyecto, que explícitamente no incluyó entrenar redes neuronales desde cero en un conjunto propio de gran escala.

El trabajo culminó con un producto operativo que puede ejecutarse en entorno local y mediante contenedores, y además se desplegó en la nube sobre Google Cloud Platform (GCP), de modo que el acceso al sistema no se limitó a una única máquina de desarrollo. La arquitectura y el procedimiento de despliegue quedaron documentados para facilitar extensiones posteriores, como validación prospectiva frente a lecturas de referencia o adaptaciones institucionales sujetas a marcos éticos y regulatorios propios de cada jurisdicción.

1.3. Estado del arte

La revisión sistemática de Pecoraro, Marsili, Espay, Bologna y di Biase sobre tecnologías de visión por computadora en trastornos del movimiento resume decenas de estudios entre 1984 y 2024 [2]. En ese conjunto de investigaciones predominan trabajos sobre la enfermedad de Parkinson y tareas de marcha. Para la estimación de pose sin marcadores físicos aparecen con frecuencia implementaciones basadas en OpenPose y, en menor medida, otras soluciones mediante modelos propios o integraciones desarrolladas para cada estudio.

Los autores destacan que el análisis automático de video alcanzó en muchos informes exactitudes diagnósticas superiores al 80 % frente a etiquetas clínicas, con buena concordancia frente a acelerometría en temblor y distonía, aunque advierten heterogeneidad en condiciones de grabación y en el software empleado, lo que dificulta comparar resultados entre sitios.

En paralelo, el trabajo de Sibley, Girges, Hoque y Foltynie analiza retos metodológicos del análisis de video para cuantificar la severidad motora en Parkinson y el papel de la inteligencia artificial y el aprendizaje automático en ese escenario [1]. Concluyen que la evaluación por video es accesible y prometedora, pero que hacen falta muestras amplias y diversas y estándares de desempeño alineados con la práctica clínica antes de generalizar su uso.

Para la estimación de pose en tiempo casi real sobre hardware común, Google publicó el marco MediaPipe, que permite encadenar módulos de visión reutilizables y ajustar el consumo de recursos según el equipo disponible [3]. Dentro de ese ecosistema, el modelo de pose corporal ofrece coordenadas normalizadas de puntos anatómicos en dos y tres dimensiones y se utiliza en prototipos de salud y deporte por su equilibrio entre latencia y precisión aceptable para seguimiento de extremidades.

Entre las implementaciones recientes con fines clínicos y de investigación, VisionMD constituyó una plataforma de código abierto destinada al análisis automático de videos de tareas de la parte III de la escala MDS-UPDRS [4]. Mediante aprendizaje profundo localizó al sujeto, permitió marcar con precisión de fotograma el inicio y el fin de cada prueba y derivó variables cinemáticas como: amplitud, velocidad y variabilidad a partir del seguimiento markerless de mano y cuerpo con el marco MediaPipe [4, 3]. El procesamiento se ejecutó en equipos convencionales sin requerir hardware especializado ni envío obligatorio de los videos a servicios en la nube, en contraste con plataformas comerciales cerradas señaladas en la misma publicación.

En estudios piloto con pacientes con enfermedad de Parkinson, los autores reportaron diferencias significativas en métricas de finger tapping y otras tareas frente a estados de estimulación cerebral profunda encendida o apagada y frente a niveles de medicación, junto con concordancia inter-evaluador elevada en el uso de la herramienta [4]. Ese trabajo ilustró la viabilidad de cuantificar de forma objetiva pruebas ya incorporadas a la exploración motora estandarizada. El prototipo documentado en esta memoria se inscribió en la misma línea de visión markerless basada en MediaPipe Pose y en el análisis de tareas motoras finas a partir de series de keypoints, pero priorizó métricas de oscilación, un pipeline orientado a servicios y despliegue mediante contenedores, como se describe en los capítulos siguientes.

En la tabla 1.1 se contrastan enfoques representativos del estado del arte en cuanto a insumos, salida típica y forma habitual de evaluación reportada en la literatura de revisión. No constituye un inventario exhaustivo de productos comerciales sino una guía conceptual para situar el trabajo desarrollado.

TABLA 1.1. Enfoques frecuentes para cuantificar el movimiento en Parkinson y contextos afines.

Enfoque	Datos de entrada	Salida típica	Evaluación habitual en la literatura
Escalas clínicas (MDS-UPDRS III y otras).	Observación en vivo o video asistido por humano.	Puntuaciones ordinales por ítem.	Consistencia inter-examinador, validez frente a estándares clínicos.
Sensores inerciales.	Aceleración o velocidad angular en segmentos corporales.	Series temporales, estimadores de frecuencia y amplitud.	Concordancia con video o con escala clínica.
OpenPose u otras redes de pose markerless.	Video monocular o multivista.	Coordenadas 2D o 3D de articulaciones.	Exactitud de clasificación o correlación con medidas de referencia [2].
MediaPipe Pose.	Video; modelos pre-entrenados integrados al marco.	Landmarks normalizados 2D/3D y continuidad temporal en un grafo de procesamiento integrado [3].	Idoneidad para tiempo casi real y bajo consumo en dispositivos habituales, frente a estimadores más pesados [3].

1.4. Objetivos del trabajo

Con el contexto clínico, la motivación del análisis automático de video y el panorama de la literatura ya expuestos, a continuación se enuncian el objetivo general y los objetivos específicos que guiaron el desarrollo del sistema.

1.4.1. Objetivo general

Diseñar e implementar un sistema de análisis digital de oscilaciones y desempeño motor en videos de ejercicios clínicos asociados a la enfermedad de Parkinson, apoyado en estimación automática de pose y en técnicas de procesamiento de señales e inteligencia artificial, que entregue métricas cuantitativas y visualizaciones comprensibles para apoyar el seguimiento objetivo del movimiento.

1.4.2. Objetivos específicos

1. Implementar un flujo de ingestión y preprocesamiento de video que alimente de manera estable el estimador de pose y módulos posteriores.
2. Integrar un modelo preentrenado de estimación de pose corporal y un seguimiento temporal de puntos relevantes para los ejercicios considerados.

3. Calcular métricas de amplitud, frecuencia, regularidad y estabilidad postural, u otras derivadas del mismo principio de análisis de series articulares, y agregarlas por sesión para su revisión estadística.
4. Exponer los resultados mediante una interfaz de servicios y una aplicación web que permitan cargar sesiones, inspeccionar gráficos y exportar reportes estructurados.
5. Verificar el comportamiento del software mediante pruebas unitarias y de integración sobre componentes críticos del flujo de procesamiento.

Los criterios de cumplimiento asociados a estos objetivos fueron la ejecución exitosa del flujo completo sobre videos de prueba disponibles para el proyecto, la coherencia de las métricas frente a escenarios controlados o simulados descritos en el capítulo de ensayos, y el paso de la batería de pruebas automatizadas definida en el repositorio del sistema.

Capítulo 2

Introducción específica

En este capítulo se describen las herramientas, bibliotecas, modelos preentrenados y conjuntos de datos provistos por terceros que sostuvieron el desarrollo del sistema. También se detallan el origen y las características de los videos utilizados y la infraestructura de ejecución contemplada para los ensayos y para el despliegue del prototipo.

2.1. Frameworks y bibliotecas

El sistema se construyó sobre tecnologías de código abierto. El núcleo del trabajo es un flujo de procesamiento de video y señales escrito en `Python 3.12`, sobre el que se montan un servicio HTTP, una capa de persistencia y una interfaz web. Esta separación permitió que la misma lógica de procesamiento se pudiera invocar desde una línea de comandos, un servicio HTTP o un contenedor sin alterar el núcleo del cálculo.

Para la lectura y manipulación de imagen se utilizó `OpenCV` [5], que cubrió la apertura de los archivos de video, la conversión entre espacios de color y las operaciones de redimensionado y muestreo cuadro a cuadro. La extracción del canal de audio asociado a las grabaciones se realizó con `moviepy`, y el análisis posterior de ese canal, en particular para la detección de los pulsos del metrónomo que estructuran los ejercicios, se apoyó en `librosa` [6], una biblioteca especializada en el procesamiento de señales de audio musicales y rítmicas.

El cálculo de las métricas se sostuvo en el conjunto de bibliotecas científicas habituales del ecosistema `Python`, integrado por `NumPy`, `SciPy` y `pandas` [7]. `NumPy` proveyó las operaciones vectoriales sobre las series temporales de los puntos detectados, `SciPy` aportó los estimadores espectrales y los algoritmos de detección de picos utilizados para el cálculo de frecuencia y de regularidad, y `pandas` se utilizó para organizar los datos por cuadro en estructuras tabulares que luego alimentan los pasos de agregación y exportación.

La capa de servicios se desarrolló con `FastAPI` [8], un framework para servicios HTTP en `Python` con soporte nativo para validación de datos a través de `Pydantic`, generación automática de documentación interactiva y eventos enviados desde el servidor (server-sent events) que se utilizaron para informar el progreso de cada análisis en tiempo casi real. La persistencia de pacientes, sesiones y resultados quedó a cargo de `PostgreSQL` [9], una base de datos relacional de uso amplio en aplicaciones de producción, accedida desde la aplicación mediante el object-relational mapper `SQLAlchemy` y administrada mediante `Alembic` para el versionado del esquema.

La interfaz de usuario se construyó como una aplicación web de página única en `TypeScript`, con un conjunto estándar de bibliotecas para componentes visuales, ruteo y gráficos interactivos. El detalle de esa capa se desarrolla en el capítulo de diseño e implementación.

En el plano de operación y empaquetado se adoptaron `Docker` [10] y `Docker Compose` para orquestar la base de datos, el servicio HTTP y un servidor web de archivos estáticos en una topología única reproducible en el entorno local y en el destino de despliegue. Las dependencias de `Python` se gestionaron con `uv`, mientras que la calidad del código se verificó mediante análisis estático y un conjunto de pruebas automatizadas con `pytest`.

En la tabla 2.1 se listan las bibliotecas y herramientas más relevantes para el procesamiento de datos y para el funcionamiento del sistema, agrupadas por su rol y con la motivación principal de su elección.

TABLA 2.1. Bibliotecas y herramientas de terceros utilizadas en el desarrollo del sistema, agrupadas por capa funcional.

Capa	Herramientas	Motivo de elección
Visión y video	MediaPipe, OpenCV	Modelos preentrenados de pose listos para uso y manipulación robusta de archivos de video.
Audio	librosa, moviepy	Extracción del canal de audio y detección de pulsos rítmicos para los ejercicios con metrónomo.
Numérica y métricas	NumPy, SciPy, pandas	Operaciones vectoriales, estimadores espectrales y manipulación tabular ampliamente probadas.
Servicios HTTP	FastAPI, Pydantic	Validación de datos integrada y soporte nativo para flujos de progreso por SSE.
Persistencia	PostgreSQL, SQLAlchemy, Alembic.	Base relacional madura, ORM y versionado declarativo del esquema.
Modelo de lenguaje	OpenAI Python SDK	Acceso a un modelo de lenguaje generativo para resúmenes interpretativos de las sesiones.
Operación	Docker, Docker Compose, uv	Despliegue equivalente en local y nube y dependencias reproducibles.

La elección de tecnologías favoreció componentes de uso difundido y bien documentados, lo que se alineó con los supuestos del trabajo sobre disponibilidad de bibliotecas de código abierto y contribuyó a mitigar el riesgo identificado en la planificación inicial respecto de posibles incompatibilidades en la integración de modelos de visión por computadora.

2.2. Datasets y datos

El trabajo no contempló el entrenamiento de modelos desde cero, por lo que las necesidades de datos se concentraron en disponer de una cantidad de video suficiente para verificar la integración del flujo de procesamiento, evaluar la coherencia de las métricas en escenarios reproducibles y exponer la herramienta a movimientos representativos de los ejercicios contemplados.

No se incorporaron conjuntos de datos públicos. La conformación de un conjunto de grabaciones de pacientes con enfermedad de Parkinson exige un marco de consentimiento informado, anonimización y resguardo bajo regulaciones de protección de datos de salud, condiciones que se inscriben en una etapa posterior a la de un prototipo de investigación. Esa restricción se reflejó desde la planificación inicial, que excluyó explícitamente del alcance la validación clínica formal y la integración con sistemas hospitalarios y dejó la evaluación funcional con especialistas como actividad opcional.

En el desarrollo se realizó además un análisis preliminar de los principales puntos de mejora y de criticidad para que el sistema pueda alinearse con normativas vigentes de protección de datos sanitarios, como la *Health Insurance Portability and Accountability Act* (HIPAA) en su jurisdicción de origen. El resultado de ese análisis se desarrolla en los capítulos posteriores como trabajo futuro y propuestas de mejora.

Para sostener el avance del desarrollo dentro del alcance descripto, se trabajó con dos fuentes complementarias de video: grabaciones propias de personas sin patologías motoras que reprodujeron los ejercicios estandarizados y un conjunto reducido de grabaciones de pacientes obtenidas en condiciones controladas y bajo acuerdo de uso interno, que se mantuvieron por fuera del repositorio de código.

Esta estrategia resultó adecuada para los objetivos del trabajo porque el sistema se apoya en modelos preentrenados de estimación de pose, cuya capacidad de detectar puntos anatómicos no depende del diagnóstico de la persona registrada. Las grabaciones de personas sin patologías motoras resultaron suficientes para verificar la robustez del seguimiento de keypoints bajo distintos encuadres y condiciones de iluminación, mientras que las grabaciones de pacientes permitieron observar el comportamiento del sistema en presencia de los patrones motores característicos. La ampliación posterior del conjunto con material clínico anotado se identificó como una línea natural de continuidad y se desarrolla en el capítulo de conclusiones.

Las grabaciones se obtuvieron con la cámara posterior de un teléfono iPhone 15 Pro a 30 cuadros por segundo, en disposición fija sobre una superficie plana y con la región de interés (mano dominante o tronco superior, según el ejercicio) centrada en el cuadro. Durante los primeros segundos de cada grabación se ubicó en el campo visual un tablero de calibración ChArUco [11], que el sistema utiliza para estimar la conversión píxeles–milímetros sin requerir intervención manual del operador. En todas las fuentes del conjunto se registraron los mismos ejercicios: finger tapping, temblor de mano y oscilación de brazos. En la tabla 2.2 se resumen las características de cada fuente de video y las condiciones de captura asociadas.

TABLA 2.2. Caracterización resumida del conjunto de grabaciones de video utilizado en el desarrollo y la verificación del sistema.

Fuente	Notas de captura
Grabaciones propias de personas sin patologías motoras.	Cámara fija, iluminación ambiente, tablero de calibración al inicio.
Grabaciones reducidas de pacientes.	Obtenidas bajo acuerdo de uso interno, almacenadas por fuera del repositorio.
Material de muestra anonimizado.	Incluido en el repositorio para pruebas de integración continua y para la incorporación de nuevos colaboradores.

El conjunto se administró bajo una clasificación de tres niveles definida en el repositorio del sistema: material de muestra anonimizado, grabaciones reales de pacientes y artefactos de salida del análisis, con políticas explícitas de almacenamiento y de exclusión del control de versiones para los dos últimos. Esa política se documentó internamente para evitar la difusión accidental de datos sensibles y se alinea con los requerimientos éticos enunciados en la planificación inicial.

2.3. Modelos preentrenados

Una de las decisiones de alcance establecidas al inicio del trabajo fue no entrenar redes neuronales desde cero, sino apoyarse en modelos preentrenados de uso difundido. Esa decisión se fundamentó en el equilibrio entre el costo de obtener un conjunto etiquetado de gran tamaño y la disponibilidad de soluciones maduras para la estimación de pose de mano y de cuerpo completo, ya validadas en aplicaciones de salud, deporte y realidad aumentada.

Para la estimación de pose se utilizaron dos modelos del marco *MediaPipe* [3], distribuidos a través de la API de tareas (Tasks API) en su versión 0.10. El modelo *HandLandmarker* [12] entrega 21 puntos por mano detectada, organizados en falanges y articulaciones, y se utilizó en todos los ejercicios centrados en la mano. El modelo *PoseLandmarker* en su variante lite [13] entrega 33 puntos del cuerpo y se reservó para los ejercicios que requerían el contexto del torso y los brazos. Ambos modelos se descargaron desde el repositorio público de *MediaPipe* en formato float16 y se almacenaron localmente en una caché versionada de modelos.

En la figura 2.1 se muestra un ejemplo de los 21 puntos de referencia entregados por el modelo *HandLandmarker*, ubicados sobre la imagen de una mano en la postura típica del ejercicio de finger tapping.

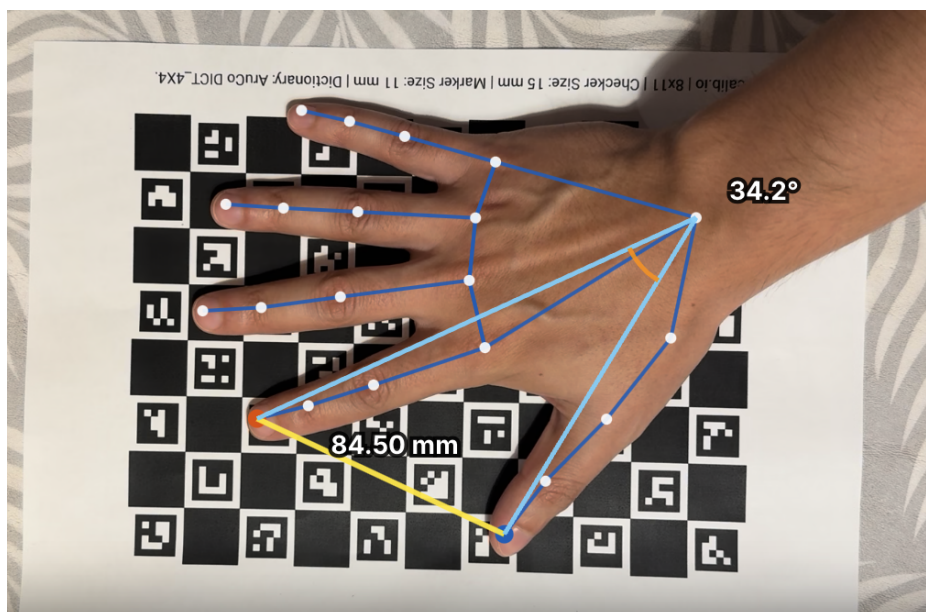


FIGURA 2.1. Ejemplo ilustrativo de los puntos de referencia entregados por el modelo HandLandmarker de MediaPipe.

En la imagen se observa la disposición regular de los puntos a lo largo de las falanges y las articulaciones, que permite reconstruir la geometría de la mano cuadro a cuadro y, a partir de ella, derivar las distancias y los ángulos sobre los que se calculan las métricas del sistema.

Para la generación automática de resúmenes interpretativos sobre los resultados de cada sesión y sobre la evolución longitudinal de un paciente se incorporó un modelo de lenguaje de gran tamaño (*large language model*, LLM), accedido a través de la API de OpenAI [14].

El modelo elegido fue GPT-5.4-mini, una variante orientada a tareas conversacionales con costo y latencia acotados. El componente del sistema responsable de esta funcionalidad envía únicamente las métricas cuantitativas y los metadatos no identificatorios de la sesión, junto con instrucciones en formato de prompt estructurado. Luego, el sistema recibe un texto en lenguaje natural que el profesional de salud puede inspeccionar como apoyo a la lectura del reporte. El modelo nunca recibe identificadores del paciente, en línea con la política de manejo de datos del trabajo.

En la tabla 2.3 se listan los modelos preentrenados utilizados, con su origen, la tarea sobre la que operan y las características relevantes para el sistema.

TABLA 2.3. Modelos preentrenados de terceros incorporados al sistema.

Modelo	Tarea	Origen y datos de pre-entrenamiento	Uso en el sistema
HandLandmarker (MediaPipe 0.10)	Detección de 21 puntos por mano.	Imágenes anotadas de manos en distintas poses, entornos y condiciones de iluminación [12].	Ejercicios de finger tapping y temblor de mano.
PoseLandmarker Lite (MediaPipe 0.10)	Detección de 33 puntos corporales.	Imágenes y video con anotaciones de pose corporal completa [13].	Ejercicios con con-texto de tronco y brazos.
GPT-5.4-mini (OpenAI)	Modelo de lenguaje generativo.	Conjunto general empleado por OpenAI para sus modelos de propósito general [14].	Resúmenes interpretativos por sesión y por evolución a partir de las métricas calculadas.

La integración de los modelos respetó los supuestos del proyecto sobre acceso a soluciones de código abierto y la decisión explícita de no entrenar desde cero. De ese modo se concentró el esfuerzo en el flujo de procesamiento, en la derivación de métricas y en la presentación clínica de los resultados, en lugar de en la construcción de un dataset de entrenamiento y de un protocolo de ajuste de redes neuronales. Otro aporte significativo del trabajo fue la operación e integración de los servicios necesarios para conformar una solución completa, que articula la infraestructura de ejecución, la interfaz de usuario, el servicio de aplicación y la persistencia en una arquitectura coherente y reproducible, descrita en el capítulo de diseño e implementación.

2.4. Infraestructura

Además de las bibliotecas de aplicación, el trabajo contempló tecnologías de infraestructura de terceros para empaquetar el sistema, ejecutarlo en entorno local y desplegarlo en la nube. En esta sección se caracterizan esas plataformas. La topología adoptada y el procedimiento de despliegue se desarrollan en el capítulo de diseño e implementación.

`Docker` [10] es una plataforma de contenedores que empaqueta una aplicación y sus dependencias en una imagen inmutable, ejecutable de forma aislada en sistemas operativos compatibles. `Docker Compose` permite declarar varios servicios interconectados, sus redes y volúmenes persistentes, y levantar la pila completa de manera coordinada. Es un patrón habitual para reproducir entornos de desarrollo sin instalaciones manuales.

Los modelos de estimación de pose son demandantes de memoria RAM pero, en el uso típico de `MediaPipe`, no requieren unidad de procesamiento gráfico dedicada. Debido a eso, la infraestructura prevista se orientó a cómputo en CPU con memoria ampliada, tanto en estaciones locales como en instancias gestionadas en la nube.

Para el despliegue en la nube se seleccionó *Google Cloud Platform* (GCP), ecosistema que admite ejecutar las mismas imágenes de contenedor definidas para el

entorno local. En la tabla 2.4 se resumen los servicios gestionados seleccionados para la prueba de concepto de bajo costo en una única región.

TABLA 2.4. Servicios gestionados de GCP seleccionados para la prueba de concepto.

Servicio	Descripción y rol
Cloud Run	Contenedores HTTP serverless con escalado automático, memoria y tiempos de respuesta configurables para cargas largas.
Cloud SQL	PostgreSQL administrado, el Auth Proxy habilita conexión segura sin credenciales embebidas.
Cloud Storage	Objetos duraderos (videos, resultados, modelos) en buckets con prefijos lógicos.
Secret Manager	Secretos versionados inyectados en ejecución.
Cloud Build	Construcción de imágenes y despliegue automatizado hacia Cloud Run.

Las herramientas para el despliegue local (Docker, Docker Compose, nginx) y los servicios de GCP (Cloud Run, Cloud SQL, Cloud Storage, Secret Manager, Cloud Build) cubren cómputo, persistencia, archivos y credenciales de forma portable y gestionada. Configuraciones de mayor exigencia operativa o normativa quedaron fuera del alcance de este trabajo.

Capítulo 3

Diseño e implementación

En este capítulo se describen los criterios de diseño del sistema, el flujo de procesamiento de video desde la ingestión hasta la exportación de métricas, la arquitectura analítica, la orquestación de inferencia y el cálculo de métricas, y el desarrollo de la aplicación web con su despliegue. Las bibliotecas, modelos y servicios de terceros ya presentados en el capítulo anterior se asumen como base tecnológica y aquí se detalla cómo se organizan en módulos propios del trabajo.

3.1. Pipeline de datos

El núcleo del sistema se implementó como un paquete `pipeline` en Python, independiente de la capa HTTP, de modo que el mismo flujo puede ejecutarse desde la línea de comandos o desde el servicio en segundo plano de la API. El diseño priorizó la modularidad por etapas, la reproducibilidad de parámetros mediante configuración centralizada y la calibración automática de escala sin intervención manual del operador cuando el video incluye un tablero ChArUco al inicio de la grabación, tal como se documentó en la sección [2.2](#).

3.1.1. Ingestión de video y audio

La etapa de ingestión comienza con `VideoLoader`, que abre el archivo mediante OpenCV, extrae metadatos (resolución, cuadros por segundo, cantidad de fotogramas) y expone un iterador de fotogramas con muestreo configurable. En paralelo, `AudioExtractor` decodifica el canal de audio del contenedor de video a una señal monofónica en formato de punto flotante, requisito para detectar los pulsos del metrónomo que estructuran varios protocolos de grabación.

Sobre esa señal se aplica `detect_impulses`, que enfatiza el rango de frecuencias configurado (por defecto entre 400 y 7000 Hz), calcula la envolvente y localiza picos con un umbral de prominencia adaptativo basado en el rango intercuartílico de la envolvente. Los instantes detectados se almacenan como eventos de impulso con marca temporal. A partir de ellos, `estimate_metronome_bpm` estima los pulsos por minuto del metrónomo y una medida de confianza derivada de la variabilidad entre intervalos, información que luego se incorpora al resumen de sesión para contextualizar el ritmo esperado del ejercicio.

3.1.2. Preprocesamiento de imagen

Cada fotograma leído pasa por `FrameProcessor`, que convierte el espacio de color de BGR a RGB, redimensiona el ancho a un máximo de 640 píxeles cuando

el video supera esa resolución y registra indicadores de calidad por cuadro. En el servicio HTTP el procesamiento se ejecuta en modo streaming: se lee un fotograma, se redimensiona, se estima la pose y se descarta el buffer, de modo que no se acumula en memoria la secuencia completa. Para controlar el costo computacional en videos de 30 o más cuadros por segundo, el worker subsamplea la secuencia a un máximo efectivo de 15 cuadros analizados por segundo y calcula el paso entre fotogramas como el cociente entero entre la tasa nominal y ese tope. La tasa efectiva resultante se utiliza en todas las métricas que dependen de la frecuencia de muestreo, en particular el análisis espectral del temblor.

3.1.3. Calibración espacial

Las distancias articulares se expresan preferentemente en milímetros. Para ello se estima el factor de conversión píxeles a milímetros a partir de detecciones del tablero ChArUco en una muestra de fotogramas del inicio del video (`estimate_scale_from_video_sample`). La geometría del tablero impreso (cuadrícula 5×7, lado de cuadrado 25 mm, marcadores ArUco `DICT_4X4_50`) coincide con la documentación interna del repositorio y con las condiciones de captura descritas en la sección 2.2. Cuando la tasa de detección de esquinas es insuficiente, el sistema recurre a una escala de respaldo definida en configuración o permite forzar la calibración manual heredada (`Calibration.from_pixel_distance`) mediante parámetros de distancia en píxeles y longitud real en milímetros. El muestreo de fotogramas para ChArUco se concentra en los primeros 12 s del video y en índices de cuadro acotados, con paso configurable entre muestras, de forma de privilegiar el intervalo en el que el tablero permanece visible antes del inicio del ejercicio dinámico. Cuando la detección de esquinas internas del tablero falla pero los marcadores ArUco individuales son visibles, se aplica un estimador alternativo basado en la longitud conocida del lado del marcador impreso, lo que incrementa la robustez en grabaciones con oclusión parcial del patrón.

El resultado de la calibración se propaga al seguimiento y queda registrado en el resumen de sesión: método empleado (`charuco` o `legacy`), milímetros por píxel, tasa de detección, uso de respaldo y parámetros geométricos del tablero. Esa trazabilidad permite al operador de la interfaz interpretar si las distancias en milímetros deben tomarse con reserva y habilita la superposición posterior de marcadores sobre el video almacenado para auditoría visual del procedimiento.

En la figura 3.1 se puede observar un fotograma del video con el tablero detectado para estimar la relación milímetros por píxel. Cada uno de los recuadros verdes representa la detección de los marcadores del tablero. A partir de esa medición en píxeles y conociendo la dimensión real de los marcadores en milímetros, se puede obtener la relación milímetros por píxel buscada.

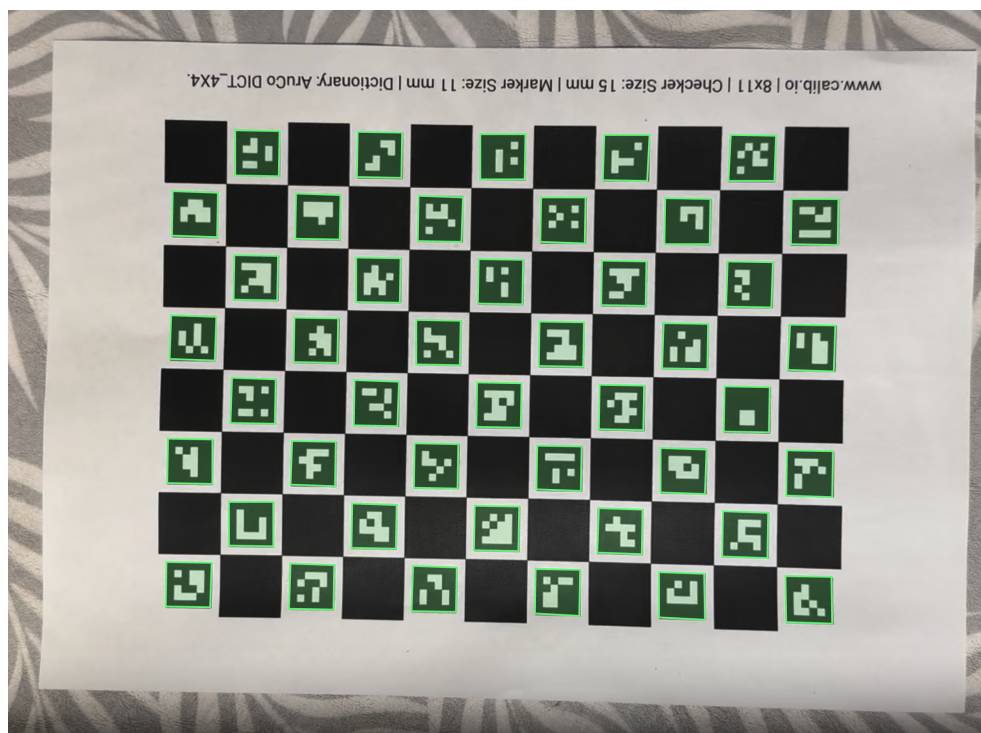


FIGURA 3.1. Detección del tablero ChArUco para estimación de relación milímetros por píxel.

3.1.4. Fase estática y segmentación del ejercicio

En el ejercicio de finger tapping, los primeros segundos de muchas grabaciones corresponden a la mano en reposo sobre el tablero de calibración. Para excluir ese tramo del cálculo de métricas dinámicas se implementó `detect_static_calibration_phase`, que calcula la desviación estándar móvil de la distancia pulgar-índice y marca como fase estática los intervalos en los que esa variabilidad permanece por debajo de 1,5 mm durante al menos 2 s. La función `slice_dynamic_after_static` recorta la serie temporal a partir del final de esa fase. Opcionalmente, en la API, el parámetro `auto_crop_taps` acota además la ventana al intervalo entre el primer y el último pico de tapping detectado en la señal, con un margen temporal de 0,1 s a cada lado.

En la figura 3.2 se observa una gráfica de la distancia pulgar-índice en función del tiempo, en donde se puede observar la fase estática y el inicio del segmento analizado para las métricas de tapping. Las líneas verticales rojas representan las detecciones de los sonidos del metrónomo que el paciente debe seguir con los movimientos del ejercicio a evaluar.

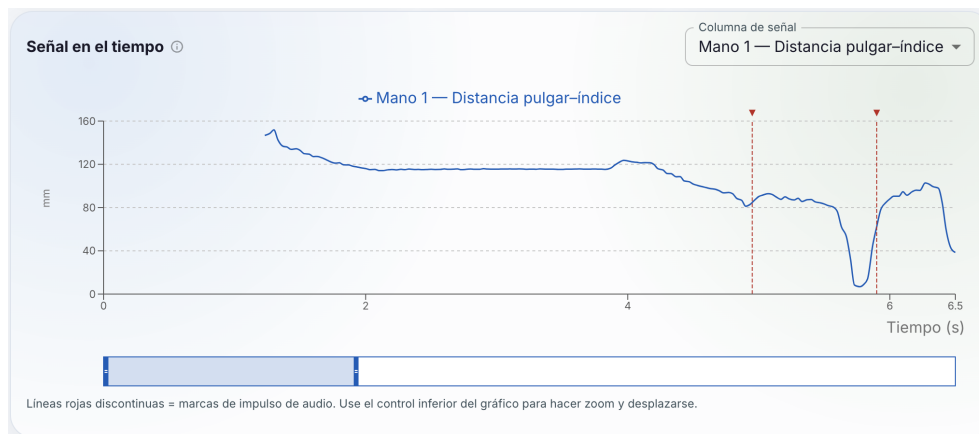


FIGURA 3.2. Serie temporal de la distancia pulgar-índice en una sesión de finger tapping.

En la tabla 3.1 se resumen las etapas del flujo de procesamiento, los datos que recibe y produce cada una y los módulos del repositorio que implementan la transformación.

TABLA 3.1. Etapas del pipeline de procesamiento de video y componentes asociados.

Etapas	Entrada	Salida	Módulo principal
Ingestión	Archivo de video.	Metadatos, audio mono, lista de impulsos.	VideoLoader, AudioExtractor.
Preprocesamiento	Fotogramas BGR.	Fotogramas RGB redimensionados.	FrameProcessor.
Calibración	Muestra de fotogramas iniciales.	Factor mm/px y objeto Calibration.	charuco, Calibration.
Estimación de pose	Fotograma procesado.	PoseResult por cuadro.	PoseEstimator.
Seguimiento	Secuencia de PoseResult.	DataFrame por fotograma.	KeypointTracker.
Segmentación	Serie temporal completa.	Subserie para métricas (tapping).	static_phase; recorte opcional.
Métricas	Serie de distancia pulgar-índice.	Lista de MetricResult.	pipeline.metrics.
Agregación y exportación	Tracking y métricas.	CSV, JSON, resumen de sesión.	SessionAggregator; escritores.

Las etapas de estimación de pose, seguimiento y segmentación se detallan en la sección 3.2, mientras que ingestión, preprocesamiento y calibración corresponden al flujo documentado en esta sección.

3.2. Arquitectura de la solución analítica

Esta sección describe cómo se relaciona la inferencia con modelos ya presentados en el capítulo 2, la construcción de series temporales a partir de landmarks y las magnitudes cuantitativas derivadas para cada sesión. Se omite el detalle de

arquitecturas y pesos de terceros, el foco recae en la orquestación propia, el procesamiento de señales unidimensionales y la agregación orientada a la exploración clínica.

3.2.1. Alcance del aprendizaje automático

La arquitectura implementada se organizó como una cadena de módulos `Python` que orquestan inferencia con modelos preentrenados de visión, transforman las salidas en series temporales tabulares y calculan métricas en señales unidimensionales. En ese marco recayeron el aporte principal del capítulo en materia de arquitectura: los contratos de datos entre etapas, el subsampleo temporal, la calibración, la agregación de sesión y el ajuste de parámetros de procesamiento de señal, umbrales de detección y reglas de segmentación documentadas en las subsecciones siguientes.

El alcance incluyó también un modelo de lenguaje de gran tamaño (LLM) como capa interpretativa sobre resultados ya cuantificados [14], orientado a resúmenes por sesión y por evolución longitudinal. El servicio `/api/insights` consume exclusivamente el `summary.json` de sesiones finalizadas y no reenvía video ni coordenadas de landmarks al proveedor externo. Las instrucciones se versionaron en plantillas YAML (`session_summary.yaml` y `evolution_summary.yaml`) con rol de sistema y plantilla de usuario separados, de modo que el equipo pudiera iterar el texto sin modificar el código del enrutador.

Durante la implementación se evaluaron varias variantes de prompt para acotar tono y extensión de la respuesta generada. Se fijó temperatura baja, límites de tokens de salida, secciones con encabezados predefinidos y prohibición explícita de formular diagnósticos, con recordatorio de que la interpretación final debe ser validada por el personal médico. Las variables interpoladas en cada solicitud se limitaron a metadatos de sesión, indicadores de calidad de la captura y tablas de métricas agregadas, sin nombres ni grabaciones del paciente, en línea con la política de manejo de datos del trabajo. El acceso al endpoint exige autenticación y verificación de que la sesión pertenece al profesional solicitante antes de invocar al modelo.

3.2.2. Criterios de diseño y configuración

La configuración global se centralizó en la clase `Settings`, que se carga con prefijo de variables de entorno `DAGO_` y valores por defecto versionados en el repositorio. De ese modo se evitó codificar rutas, umbrales de audio o geometría del tablero ChArUco dentro de la lógica. El tipo de ejercicio clínico (`finger_tapping`, `hand_tremor`, `nose_finger`, `arm_oscillation`) se propagó a lo largo del pipeline para etiquetar métricas y seleccionar el modelo de pose adecuado.

Se adoptó además una política de clasificación de datos en tres niveles en el repositorio: material de muestra anonimizado apto para integración continua, grabaciones reales de pacientes excluidas del control de versiones y directorios de salida de análisis ignorados por el sistema de seguimiento de cambios. Esa separación se mantuvo coherente con los requerimientos éticos de la planificación inicial y condicionó que las pruebas automatizadas y los ejemplos de documentación apuntaran únicamente al nivel público, mientras las sesiones clínicas reales se procesaron en entornos locales o en la nube con almacenamiento privado.

3.2.3. Contrato de datos entre etapas

Para mantener la trazabilidad entre módulos se definieron esquemas de datos inmutables en cada frontera. `VideoMeta` y `FrameData` describen el origen temporal de cada fotograma; `AudioData` e `ImpulseEvent` encapsulan el audio y los pulsos detectados; `ProcessedFrame` añade las transformaciones de imagen; `PoseResult` agrupa las detecciones de mano y cuerpo por cuadro; y `MetricResult` estandariza la salida de cada algoritmo de métrica con nombre, unidad, validez y metadatos opcionales. El resumen final de sesión (`SessionSummary`) consolida métricas, retardos, parámetros de calibración, metrónomo y versión del pipeline en un único documento JSON serializable, apto para persistencia relacional y para los prompts del modelo de lenguaje.

Ese contrato permite sustituir o extender un módulo sin alterar los demás siempre que respete las interfaces acordadas. Por ejemplo, un futuro estimador de pose alternativo podría devolver el mismo `PoseResult` sin modificar el seguimiento ni las métricas aguas abajo.

La figura 3.3 muestra el recorrido completo desde el video crudo hasta la persistencia y las interfaces de consulta. En el diagrama se indica de forma genérica la estimación de pose con `MediaPipe`. En la implementación, los ejercicios centrados en la mano emplean el modelo `HandLandmarker` y los que requieren contexto corporal el `PoseLandmarker` lite, según lo descrito en la sección 2.3. También se observa la separación entre el procesamiento analítico en CPU, la capa de persistencia relacional y los canales de acceso mediante API y aplicación web. Esa organización condicionó las decisiones de integración desarrolladas en la sección 3.3.

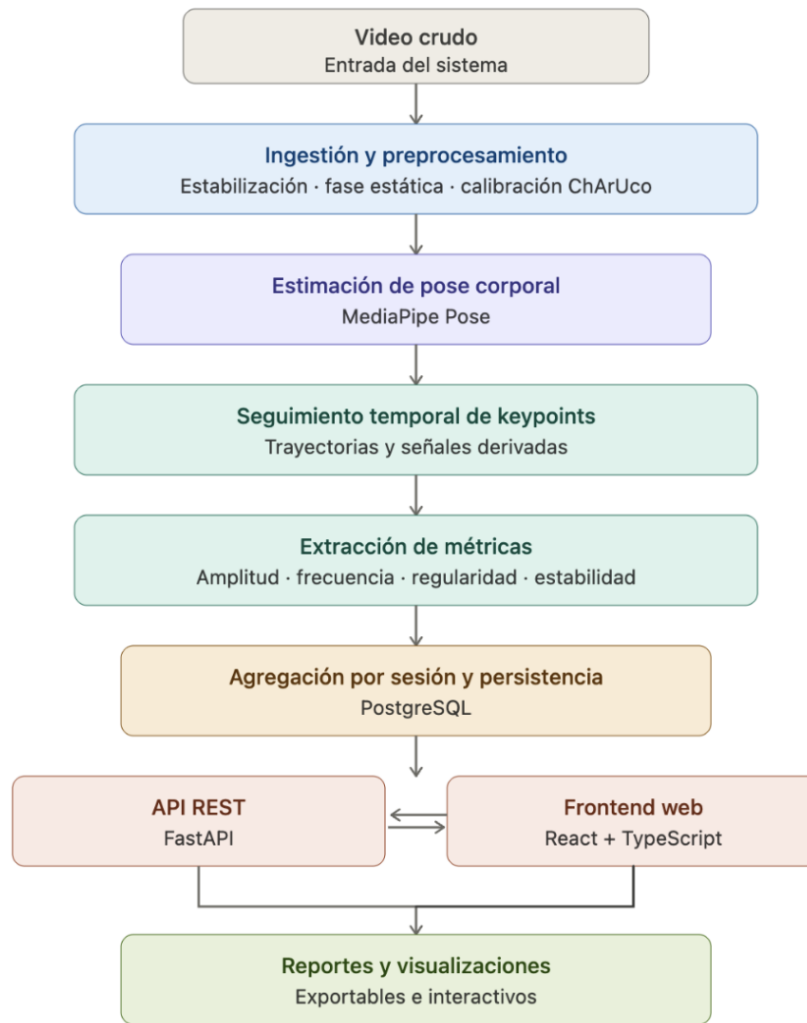


FIGURA 3.3. Flujo funcional del sistema desde la entrada de video hasta reportes y visualizaciones.

3.2.4. Orquestación de la inferencia

La función `pose_model_for_exercise` selecciona el landmarker de MediaPipe según el tipo de ejercicio: modo `HANDS` para `finger_tapping` y `hand_tremor`, y modo `HOLISTIC` para `nose_finger` y `arm_oscillation`. La clase `PoseEstimator` instancia el modelo correspondiente mediante la API de tareas, con archivos `.task` en caché local gestionados por `ModelManager`. En el servicio web se activa el modo de video del landmarker para explotar la continuidad temporal entre cuadros consecutivos.

Las coordenadas de profundidad (z) se almacenan en los esquemas internos pero no intervienen en distancias ni ángulos métricos: la cuantificación operativa permanece en dos dimensiones con escala inferida por ChArUco o calibración manual. En modo `HANDS` se retiene una sola mano dominante por fotograma; en modo `HOLISTIC` se registran además hombros, codos y muñecas para visualización.

3.2.5. Seguimiento temporal y señales derivadas

`KeypointTracker` acumula, por cada fotograma analizado, un registro tabular común con índice temporal, coordenadas normalizadas de los 21 puntos de la mano dominante, distancias derivadas en milímetros cuando existe calibración y, en modo holístico, posiciones de nariz, hombros, codos y muñecas junto con los ángulos de abducción de brazo izquierdo y derecho (`pose_left_arm_angle`, `pose_right_arm_angle`). Las detecciones ausentes se registran como valores faltantes y se rellenan hacia adelante hasta un máximo de 5 fotogramas consecutivos para atenuar oclusiones breves. Esa capa de seguimiento es compartida por los cuatro tipos de ejercicio, pero la señal que alimenta las métricas clínicas se elige en función del movimiento evaluado.

En `finger_tapping` y `hand_tremor` la señal operativa principal es la distancia pulgar-índice `hand0_thumb_index_dist`. Con factor de escala s milímetros por píxel y coordenadas normalizadas de pulgar e índice p_t y q_t en un fotograma de ancho W y alto H píxeles, se define

$$d_t = s \sqrt{W^2(p_{t,x} - q_{t,x})^2 + H^2(p_{t,y} - q_{t,y})^2}. \quad (3.1)$$

En `finger_tapping`, d_t sustenta los biomarcadores de tapping y la sincronía con el metrónomo. En `hand_tremor`, la misma serie (o el desplazamiento de la muñeca derivado de `hand0_wrist_px_*`) cuantifica la oscilación involuntaria de la mano en reposo, sin detección de ciclos de apertura y cierre.

En `nose_finger` se construye la distancia entre la yema del índice y la nariz a partir de los landmarks holísticos (`pose_nose_*` y `hand0_lm8_*`), con la misma conversión métrica s . El mínimo de esa distancia durante el alcance resume la precisión del contacto nariz-dedo. En `arm_oscillation` la magnitud central es el ángulo de abducción del brazo en el plano de la imagen, calculado en cada fotograma entre el vector hombro-codo y la línea de hombros. Las columnas de muñeca y mano quedan disponibles para supervisión visual, pero las métricas automáticas de ese ejercicio se expresan en grados sobre la serie angular.

3.2.6. Métricas de movimiento implementadas

El módulo de métricas selecciona, según el tipo de ejercicio, la señal temporal y el conjunto de algoritmos aplicables. Las funciones genéricas `oscillation_amplitude`, `tremor_frequency` y `movement_regularity` operan sobre la serie activa de cada ejercicio cuando hay al menos 4 muestras válidas. La frecuencia dominante se estima por transformada rápida de Fourier en la banda

$$f \in [1,0, 12,0] \text{ Hz}, \quad (3.2)$$

coherente con el rango de interés clínico para temblor y oscilaciones de extremidad superior [2], empleando la tasa efectiva tras el subsampleo ($f_s = \text{fps}_{\text{video}}/\text{step}$).

En `finger_tapping`, tras recortar la fase estática y, de forma opcional, la ventana entre el primer y el último tap, se aplican sobre $\{d_t\}$ los biomarcadores de `compute_finger_tapping_metrics`: tasa de taps, intervalo inter-tap medio y su coeficiente de variación, decremento de amplitud entre picos, índice de fatiga, velocidades medias de apertura y cierre, y el promedio del ángulo en la muñeca (`thumb_index_wrist_angle_mean`). La detección de picos usa

`scipy.signal.find_peaks` con prominencia proporcional al rango de la señal [1, 4]. `SessionAggregator` calcula además retardos entre impulsos del metrónomo y mínimos locales de d_t dentro de $\pm 0,35$ s (`delays.csv`).

En `hand_tremor`, sobre la serie de la mano en reposo se reportan amplitud de oscilación, frecuencia en la banda 3.2 y regularidad del movimiento. No se calculan métricas de tapping ni retardos con metrónomo.

En `nose_finger`, se cuantifican la distancia mínima y media índice-nariz, la longitud de la trayectoria de la yema del índice y un índice de precisión del alcance (cociente entre distancia mínima y longitud de trayectoria). Opcionalmente se resume la dispersión de la muñeca durante el gesto.

En `arm_oscillation`, se promedian y resumen los ángulos de abducción (`arm_angle_mean`, `arm_angle_variability`), se aplican amplitud, frecuencia y regularidad sobre la serie angular del brazo dominante y se estima la estabilidad postural del tronco (`postural_stability`) a partir del desplazamiento del centro de masa definido por hombros y caderas.

Cada magnitud se encapsula en un `MetricResult` con unidad, validez y nota cuando los datos son insuficientes. En el listado siguiente se resume el catálogo por tipo de ejercicio.

■ **Tapping de dedos y temblor de mano:**

- **Amplitud de oscilación** (`oscillation_amplitude`, mm): rango de d_t o de desplazamiento de muñeca.
- **Frecuencia de temblor** (`tremor_frequency`, Hz): pico espectral sobre la señal de mano, banda definida según 3.2.
- **Regularidad del movimiento** (`movement_regularity`, adim.): variabilidad y entropía de la oscilación de mano.

■ **Tapping de dedos:**

- **Tasa de taps** (`tap_rate`, 1/s): ritmo de taps asociado a bradicinesia.
- **Intervalo inter-tap medio** (`iti_mean`, s): tiempo medio entre taps consecutivos.
- **Coefficiente de variación del ITI** (`iti_cv`, adim.): consistencia del ritmo de tapping.
- **Decremento de amplitud** (`amplitude_decrement`, mm/s): pendiente de amplitud entre picos sucesivos.
- **Índice de fatiga** (`fatigue_index`, adim.): amplitud final respecto de la amplitud inicial en la ventana dinámica.
- **Velocidad de apertura en picos** (`peak_opening_velocity`, mm/s): velocidad media al abrir los dedos en los picos de d_t .
- **Velocidad de cierre en valles** (`peak_closing_velocity`, mm/s): velocidad media al cerrar los dedos en los valles.
- **Ángulo medio en muñeca** (`thumb_index_wrist_angle_mean`, °): apertura media pulgar-índice en la muñeca.

- **Retardo audio–movimiento** (`delays`, s): desfase entre el metrónomo y el mínimo de d_t , si hay audio disponible.
- **Prueba nariz–dedo:**
 - **Distancia mínima nariz–dedo** (`nose_finger_min_distance`, mm): mínima distancia entre la yema del índice y la nariz.
 - **Distancia media nariz–dedo** (`nose_finger_mean_distance`, mm): distancia media durante el alcance.
 - **Longitud de trayectoria** (`nose_finger_path_length`, mm): recorrido de la yema del índice.
 - **Precisión del alcance** (`nose_finger_accuracy`, adim.): cociente entre distancia mínima y longitud de trayectoria.
- **Oscilación de brazo:**
 - **Ángulo medio de abducción** (`arm_angle_mean`, °): abducción media del brazo analizado.
 - **Variabilidad del ángulo** (`arm_angle_variability`, °): rango pico a pico del ángulo de abducción.
 - **Amplitud de oscilación** (`oscillation_amplitude`, °): oscilación de la serie angular del brazo.
 - **Frecuencia de temblor** (`tremor_frequency`, Hz): frecuencia dominante del ángulo en banda definida según 3.2.
 - **Regularidad del movimiento** (`movement_regularity`, adim.): regularidad de la oscilación del brazo.
 - **Estabilidad postural** (`postural_stability`, adim.): balanceo del tronco, estimado como proxy hombro–cadera.

En la figura 3.4 se ilustra, a modo de ejemplo para `finger_tapping`, la serie d_t con picos de tapping y las detecciones de los pulsos del metrónomo marcados con líneas verticales rojas.

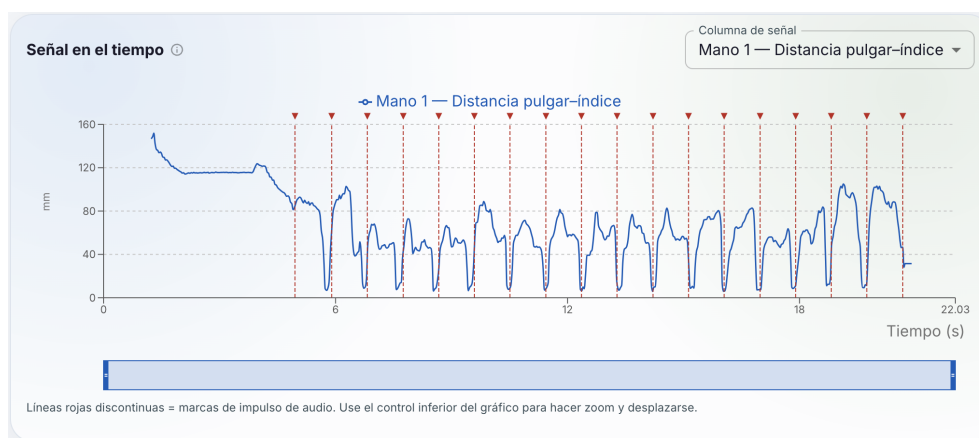


FIGURA 3.4. Serie temporal de la distancia pulgar–índice durante una sesión completa.

En la ejecución por línea de comandos se generan además gráficos `timeseries.png`, `metrics_summary.png` y `detection_overview.png` cuando la opción de guardado de figuras está activa. El worker de la API omite esos gráficos para reducir tiempo y uso de memoria en el contenedor y delega la visualización interactiva al frontend.

3.2.7. Agregación, exportación y trazabilidad

`SessionAggregator` recibe el `DataFrame` completo de seguimiento (incluida la fase estática cuando corresponde), la lista de métricas calculadas, los impulsos de audio y los metadatos de calibración. Produce un resumen con identificador de video, tipo de ejercicio, tasa de fotogramas nominal, conteo de fotogramas analizados, tabla de métricas y colección de retardos audio-movimiento. Los escritores `CsvWriter` y `JsonWriter` materializan ese contenido en disco con convenciones de nombres de columna estables, de modo que las pruebas de integración puedan verificar la existencia y el esquema de `tracking.csv`, `metrics.csv` y `summary.json` tras una ejecución de humo del comando de sesión.

La métrica de amplitud de oscilación combina rango pico a pico, media y desviación estándar de la señal centrada; la regularidad cuantifica tanto la variabilidad relativa como la entropía muestral de incrementos, conforme a la literatura de análisis de complejidad del movimiento en trastornos del movimiento [2]. La coherencia entre definiciones matemáticas en código y descripción en la memoria se verificó en el capítulo 4 mediante pruebas unitarias sobre señales sintéticas con espectro y picos conocidos.

Cada objeto `MetricResult` incluye, además del valor escalar, un indicador de validez y un campo de nota textual cuando el algoritmo no puede estimar la magnitud (señal demasiado corta, menos de 3 taps, espectro plano en la banda de temblor, entre otros casos). Esa decisión evitó presentar números engañosos en la interfaz y facilitó el diagnóstico de fallos de grabación (mano fuera de cuadro, oclusión prolongada, metrónomo inaudible). La versión del pipeline se registra en cada sesión para detectar incompatibilidades si en el futuro se modifican definiciones de métricas de forma no retrocompatible.

3.3. Integración e implementación del sistema

El prototipo operativo se estructuró en tres capas: el paquete `pipeline` como motor analítico, un servicio HTTP construido con `FastAPI` y una aplicación de página única en `React` servida mediante `nginx`. La persistencia de usuarios, pacientes y sesiones reside en `PostgreSQL`, con migraciones versionadas mediante `Alembic`, en línea con las herramientas citadas en la sección 2.1.

3.3.1. Diferencias entre ejecución por CLI y por API

Aunque ambos caminos comparten la lógica analítica, existieron diferencias operativas deliberadas en su diseño. La CLI procesa el video a la tasa de muestreo definida por el argumento `-step` o, por defecto, todos los fotogramas, mientras que el worker de la API impone el tope de 15 cuadros analizados por segundo y escribe el progreso en la cola SSE. La CLI genera figuras estáticas de control de calidad; la API prioriza la latencia y el pico de memoria del contenedor. La CLI acepta rutas de archivo locales; la API recibe bytes en memoria, los persiste en

un archivo temporal y los elimina al finalizar; conserva únicamente la copia en el directorio de salida de la sesión. Esas variantes respondieron a usos distintos: experimentación offline e integración continua frente a carga interactiva desde el navegador.

3.3.2. Servicio HTTP y procesamiento en segundo plano

La aplicación `api.main` registra routers para autenticación (`/api/auth`), pacientes (`/api/patients`), sesiones de análisis (`/api/sessions`), comparación entre sesiones (`/api/compare`), generación de informes (`/api/sessions/.../report`) e interpretación asistida por modelo de lenguaje (`/api/insights`). Los esquemas de entrada y salida se validan con modelos `Pydantic`, lo que homogeneiza los tipos entre el cliente web y el servidor y reduce errores de formato en la carga multipart de videos.

El registro de usuarios y el inicio de sesión emiten tokens JWT que deben acompañar las peticiones subsiguientes. Los roles distinguen al menos cuentas de personal de salud y cuentas de portal de paciente con acceso de solo lectura a su propia evolución, de modo de separar la carga de nuevas sesiones de la consulta longitudinal sin exponer datos de terceros.

La carga de un video se realiza mediante `POST /api/sessions/analyse`, que crea un registro de sesión en estado pendiente y encola un trabajo en un `ThreadPoolExecutor` con capacidad limitada (2 hilos concurrentes y cola acotada) para evitar saturar la memoria del host. Si la cola supera el máximo configurado, el servicio responde con código HTTP 503 e informa al cliente que debe reintentar más tarde. Existe además un endpoint de análisis por lotes que procesa varios archivos en secuencia y mantiene el orden de envío; resulta útil para sesiones de laboratorio con múltiples repeticiones del mismo ejercicio.

Cada trabajo ejecuta `run_pipeline` en un hilo separado. Durante el procesamiento, los eventos de progreso (`ingestion`, `preprocessing`, `pose_estimation`, `metrics`, `aggregation`, `done` o `error`) se publican en una cola `thread-safe` asociada al identificador de sesión. El endpoint `GET /api/sessions/{id}/status` transmite esos eventos al navegador mediante eventos enviados desde el servidor (SSE), con autenticación por token en la cadena de consulta para compatibilidad con `EventSource`. Al finalizar, un `coroutine` asíncrono actualiza el campo `summary_json` y la ruta de salida en la tabla `AnalysisSession`.

3.3.3. Manejo de errores y recuperación de sesión

Si una excepción interrumpe el pipeline en el hilo de trabajo, el estado de la sesión pasa a `error`, se registra el mensaje en la base de datos y se emite un evento SSE de error al cliente. Los archivos temporales del video subido se eliminan en un bloque `finally` para no agotar el espacio en disco del contenedor. Cuando el proceso del servidor se reinicia y pierde la cola en memoria, el endpoint de estado puede consultar el registro persistido y devolver el estado terminal (`done` o `error`) tras un tiempo de espera acotado, de modo que el operador no quede indefinidamente suscrito a un análisis ya finalizado.

3.3.4. Artefactos y consulta de resultados

Por cada análisis exitoso se escriben en un directorio temporal (luego referenciado desde la base de datos) los archivos `tracking.csv` (serie temporal por fotograma), `metrics.csv` (tabla de métricas con validez y notas), `delays.csv` (retardos respecto del metrónomo), `summary.json` (resumen agregado con metadatos de calibración y metrónomo) y una copia `input.mp4` del video original para reproducción y superposición de marcadores ChArUco. El frontend recupera el resumen y el tracking mediante peticiones REST autenticadas y ofrece descarga de los artefactos y generación de informe en PDF mediante ReportLab en el servidor.

En la tabla 3.2 se relacionan esos productos con su uso en la interfaz y en los informes.

TABLA 3.2. Artefactos generados por sesión y su consumo en la aplicación.

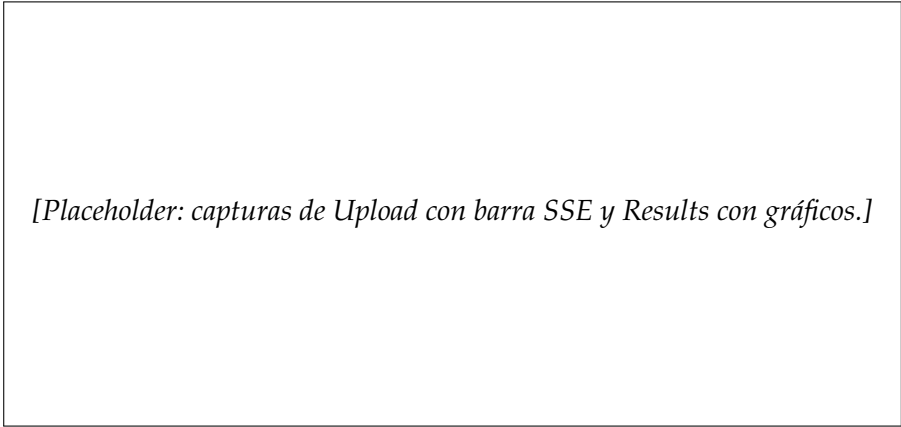
Artefacto	Formato	Uso
<code>summary.json</code>	JSON	Resumen en pantalla de resultados, evolución longitudinal y prompts al LLM.
<code>tracking.csv</code>	CSV	Gráficos interactivos de series temporales y ángulos.
<code>metrics.csv</code>	CSV	Tabla de métricas exportable e informe PDF.
<code>delays.csv</code>	CSV	Análisis de sincronización con metrónomo (cuando aplica).
<code>input.mp4</code>	Video	Reproducción con superposición de landmarks y marcadores ChArUco.
Informe PDF	PDF	Exportación para revisión offline por el profesional.

3.3.5. Interfaz web

La aplicación React define rutas para inicio de sesión y registro, carga de videos (`UploadPage`) con selección de ejercicio y parámetros de calibración, listado y detalle de pacientes, visualización de resultados por sesión (`ResultsPage`), exportación, comparación entre dos sesiones y un panel de progreso del paciente. El cliente HTTP centraliza las llamadas en `api/client.ts`, entre ellas la suscripción SSE que mapea las etapas del pipeline a mensajes legibles en la barra de progreso.

En la pantalla de resultados se combinan tablas de métricas con gráficos de series temporales contruidos con una biblioteca de gráficos declarativos, superposición opcional de landmarks sobre el video y solicitud de resúmenes interpretativos bajo demanda. La vista de detalle de paciente integra un componente de evolución que filtra sesiones previas por tipo de ejercicio y representa la tendencia de biomarcadores seleccionados a lo largo del tiempo, en apoyo del seguimiento longitudinal previsto en los objetivos del trabajo. La página de comparación envía las métricas de dos sesiones al servicio de insights para obtener un texto contrastivo, sin transmitir imágenes ni identificadores del paciente al proveedor del modelo de lenguaje.

En la figura 3.5 se espera una composición con dos capturas de pantalla: la vista de carga con progreso en tiempo real y la vista de resultados con gráficos de métricas, sin datos identificables del paciente.



[Placeholder: capturas de Upload con barra SSE y Results con gráficos.]

FIGURA 3.5. Interfaz de carga de sesión y de visualización de resultados (imagen pendiente de incorporar).

Los endpoints de interpretación envían al modelo de lenguaje únicamente métricas agregadas y metadatos no identificatorios, conforme a la política descrita en la sección 2.3, y devuelven texto en lenguaje natural como apoyo a la lectura del informe.

3.3.6. Despliegue local y en la nube

En entorno local, `docker-compose` levanta los servicios `db` (`postgres:16-alpine`), `api` (imagen multi-etapa con Python 3.12, dependencias instaladas con `uv` y límite de memoria de 8 GB) y `frontend` (`nginx` sirve el build estático y proxifica la API). Un perfil opcional `cli` permite ejecutar el mismo contenedor en modo batch con el módulo `pipeline.cli.run_session`. Los modelos MediaPipe y los videos de muestra se montan como volúmenes para no incluirlos en la imagen y facilitar la actualización.

El punto de entrada del contenedor de API ejecuta migraciones de esquema con Alembic antes de levantar el servidor `uvicorn`, de modo que un despliegue nuevo aplique automáticamente las revisiones pendientes sobre PostgreSQL. El frontend se construyó en una etapa previa de la imagen multi-etapa (Node.js para compilar TypeScript y `nginx` para servir archivos estáticos) y el runtime de inferencia se fijó en `linux/amd64` por compatibilidad con las ruedas binarias de MediaPipe en arquitecturas sin soporte oficial para brazo.

Para la prueba de concepto en la nube se replicó esa topología sobre los servicios gestionados de GCP enumerados en la tabla 2.4: Cloud Run aloja el contenedor HTTP con API y pipeline, Cloud SQL aloja PostgreSQL, Cloud Storage almacena videos y resultados, Secret Manager guarda credenciales y Cloud Build ejecuta la cadena de construcción y despliegue. La figura 3.6 resume las relaciones entre el usuario, el cómputo serverless y los servicios de datos y secretos.

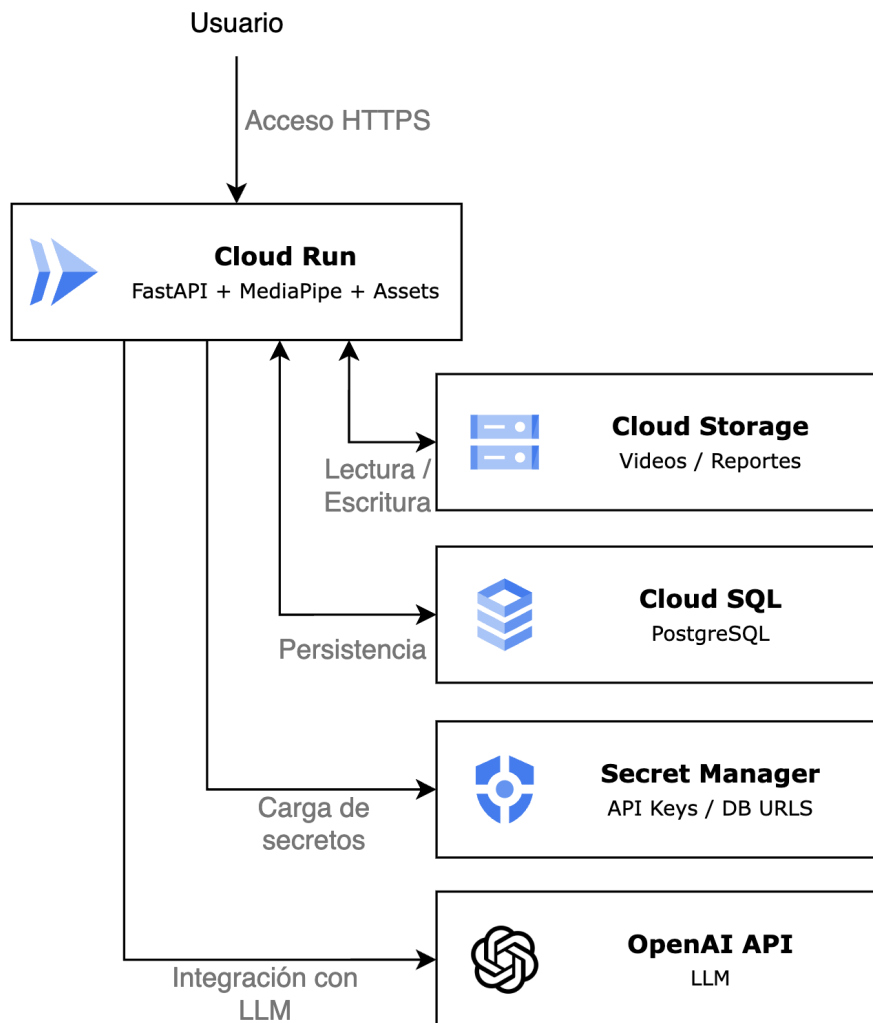


FIGURA 3.6. Arquitectura de despliegue del prototipo en Google Cloud Platform.

En el diagrama se aprecia que el procesamiento de video y la inferencia con MediaPipe residen en el mismo servicio de cómputo que expone la API REST, mientras que la persistencia estructurada y los objetos de gran tamaño se externalizan a servicios administrados. Esa separación facilitó escalar horizontalmente el acceso HTTP sin duplicar el almacenamiento de videos en cada instancia, aunque el estado de progreso SSE permanece en memoria de proceso en la versión desplegada, lo que constituye una limitación conocida para despliegues multi-réplica que se abordará en el capítulo de conclusiones.

Con la arquitectura descrita quedó cerrado el ciclo de diseño e implementación: desde la captura estandarizada descrita en el capítulo anterior hasta la consulta web de métricas y exportación de informes. La verificación automatizada del desarrollo, pruebas unitarias del pipeline e integración de humo de extremo a extremo, se documentó en el capítulo 4. El capítulo siguiente presentará el plan de evaluación y los resultados obtenidos al ejecutar este sistema sobre el conjunto de videos disponible para el trabajo.

Capítulo 4

Ensayos y resultados

En este capítulo se presentó el plan de evaluación del sistema, las pruebas automatizadas que respaldaron la reproducibilidad del pipeline implementado en el capítulo 3 y los resultados obtenidos al ejecutar el prototipo sobre los videos disponibles para el trabajo.

4.1. Pruebas automatizadas del pipeline

Durante el desarrollo se mantuvo una batería de pruebas unitarias sobre componentes aislados del flujo analítico: calibración ChArUco, detección de fase estática, impulsos de audio, métricas espectrales y de tapping, ángulos derivados y escritores de exportación. Los módulos bajo `tests/unit/` validaron comportamientos con señales sintéticas o entradas controladas, de modo que las definiciones matemáticas descritas en el capítulo 3 pudieran contrastarse con salidas esperadas. Se incluyó además una prueba de integración de humo (`tests/integration/test_cli_session.py`) que ejecutó el comando de sesión sobre un video de muestra del repositorio y verificó la generación de `tracking.csv`, `metrics.csv` y `summary.json` con esquemas estables.

Esas pruebas no sustituyeron la validación clínica formal, excluida del alcance del trabajo, pero demostraron que el ensamblado descrito en el capítulo de diseño era reproducible y detectaba regresiones en el flujo de extremo a extremo.

Capítulo 5

Conclusiones

Todos los capítulos deben comenzar con un breve párrafo introductorio que indique cuál es el contenido que se encontrará al leerlo. La redacción sobre el contenido de la memoria debe hacerse en presente y todo lo referido al proyecto en pasado, siempre de modo impersonal.

5.1. Conclusiones generales

La idea de esta sección es resaltar cuáles son los principales aportes del trabajo realizado y cómo se podría continuar. Debe ser especialmente breve y concisa. Es buena idea usar un listado para enumerar los logros obtenidos.

En esta sección no se deben incluir ni tablas ni gráficos.

Algunas preguntas que pueden servir para completar este capítulo:

- ¿Cuál es el grado de cumplimiento de los requerimientos?
- ¿Cuán fielmente se pudo seguir la planificación original (cronograma incluido)?
- ¿Se manifestó algunos de los riesgos identificados en la planificación? ¿Fue efectivo el plan de mitigación? ¿Se debió aplicar alguna otra acción no contemplada previamente?
- Si se debieron hacer modificaciones a lo planificado ¿Cuáles fueron las causas y los efectos?
- ¿Qué técnicas resultaron útiles para el desarrollo del proyecto y cuáles no tanto?

5.2. Próximos pasos

Acá se indica cómo se podría continuar el trabajo más adelante.

Bibliografía

- [1] Krista G. Sibley, Christine Girges, Ehsan Hoque y Thomas Foltynie. «Video-Based Analyses of Parkinson's Disease Severity: A Brief Review». En: *Journal of Parkinson's Disease* 11.Suppl. 1 (2021), S83-S93. DOI: [10.3233 / JPD-202402](https://doi.org/10.3233/JPD-202402).
- [2] Pasquale Maria Pecoraro, Luca Marsili, Alberto J. Espay, Matteo Bologna y Lazzaro di Biase. «Computer Vision Technologies in Movement Disorders: A Systematic Review». En: *Movement Disorders Clinical Practice* 12.9 (2025), págs. 1229-1243. DOI: [10.1002/mdc3.70123](https://doi.org/10.1002/mdc3.70123).
- [3] Camillo Lugaresi, Jiuqiang Tang, Hadon Nash, Chris McClanahan, Esha Uboweja, Michael Hays, Fan Zhang, Chuo-Ling Chang, Ming Guang Yong, Juhyun Lee, Wan-Teh Chang, Wei Hua, Manfred Georg y Matthias Grundmann. «MediaPipe: A Framework for Building Perception Pipelines». En: *arXiv preprint arXiv:1906.08172* (2019).
- [4] Gabriela Acevedo, Florian Lange, Carolina Calonge, Robert Peach, Joshua K. Wong y Diego L. Guarín. «VisionMD: An Open-Source Tool for Video-Based Analysis of Motor Function in Movement Disorders». En: *npj Parkinson's Disease* 11.1 (2025), pág. 27. DOI: [10.1038/s41531-025-00876-6](https://doi.org/10.1038/s41531-025-00876-6).
- [5] Gary Bradski. «The OpenCV Library». En: *Dr. Dobb's Journal of Software Tools* (2000).
- [6] Brian McFee, Colin Raffel, Dawen Liang, Daniel P. W. Ellis, Matt McVicar, Eric Battenberg y Oriol Nieto. «librosa: Audio and Music Signal Analysis in Python». En: *Proceedings of the 14th Python in Science Conference (SciPy)*. 2015, págs. 18-25. DOI: [10.25080/Majora-7b98e3ed-003](https://doi.org/10.25080/Majora-7b98e3ed-003).
- [7] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, Stéfan J. van der Walt, Matthew Brett, Joshua Wilson, K. Jarrod Millman, Nikolay Mayorov, Andrew R. J. Nelson, Eric Jones, Robert Kern, Eric Larson, C J Carey, İlhan Polat, Yu Feng, Eric W. Moore, Jake VanderPlas, Denis Laxalde, Josef Perktold, Robert Cimrman, Ian Henriksen, E. A. Quintero, Charles R. Harris, Anne M. Archibald, Antônio H. Ribeiro, Fabian Pedregosa y Paul van Mulbregt. «SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python». En: *Nature Methods* 17 (2020), págs. 261-272. DOI: [10.1038/s41592-019-0686-2](https://doi.org/10.1038/s41592-019-0686-2).
- [8] Sebastián Ramírez. *FastAPI*. <https://fastapi.tiangolo.com/>. Disponible: 2026-04-15. 2024.
- [9] The PostgreSQL Global Development Group. *PostgreSQL: The World's Most Advanced Open Source Relational Database*. <https://www.postgresql.org/>. Disponible: 2026-04-15. 2024.
- [10] Dirk Merkel. «Docker: Lightweight Linux Containers for Consistent Development and Deployment». En: *Linux Journal* 2014.239 (2014).
- [11] S. Garrido-Jurado, R. Muñoz-Salinas, F. J. Madrid-Cuevas y M. J. Marín-Jiménez. «Automatic generation and detection of highly reliable fiducial

- markers under occlusion». En: *Pattern Recognition* 47.6 (2014), págs. 2280-2292. DOI: [10.1016/j.patcog.2014.01.005](https://doi.org/10.1016/j.patcog.2014.01.005).
- [12] Fan Zhang, Valentin Bazarevsky, Andrey Vakunov, Andrei Tkachenka, George Sung, Chuo-Ling Chang y Matthias Grundmann. *MediaPipe Hands: On-device Real-time Hand Tracking*. arXiv preprint arXiv:2006.10214. 2020. URL: <https://arxiv.org/abs/2006.10214>.
- [13] Valentin Bazarevsky, Ivan Grishchenko, Karthik Raveendran, Tyler Zhu, Fan Zhang y Matthias Grundmann. *BlazePose: On-device Real-time Body Pose tracking*. arXiv preprint arXiv:2006.10204. 2020. URL: <https://arxiv.org/abs/2006.10204>.
- [14] OpenAI. *OpenAI API Reference*. <https://platform.openai.com/docs/api-reference>. Disponible: 2026-04-15. 2026.